

OpenDccCLib

Developer Guide — Decoder

From first project to a fully featured DCC decoder

Covers all supported platforms and IDEs

Table of Contents

1. Introduction
 - 1.1 What the Library Does (and Does Not Do)
 - 1.2 Platform Support
2. DCC Decoder Concepts
 - 2.1 Signal Reception
 - 2.2 Bit Classification
 - 2.3 Packet Structure
 - 2.4 Address Matching
 - 2.5 Configuration Variables (CVs)
3. Project File Structure
4. dcc_user_config.h in Depth
5. Initialization — decoder.c
 - 5.1 The dcc_config_t Struct (Decoder Fields)
 - 5.2 Edge Timestamp Ring Buffer
 - 5.3 Setup and Main Loop
6. ISR Architecture
 - 6.1 GPIO Edge Interrupt
 - 6.2 100ms Periodic Tick
7. Implementing the Drivers
 - 7.1 Platform Drivers
 - 7.2 CV Storage Drivers
 - 7.3 ACK Pulse Driver
8. The Bit Decoder
9. The Packet Decoder
10. CV Storage and Address Matching
11. Service Mode (Decoder Side)
12. RailCom Responses (Brief Overview)
13. Failsafe Mode
14. Callbacks — Where Your Application Lives
15. Porting to a New MCU
16. Unit Testing

17. Troubleshooting

1. Introduction

OpenDccCLib is a C library that implements the NMRA DCC protocol for microcontrollers. This guide focuses on the decoder role — receiving DCC packets from the track and acting on speed, function, accessory, and CV programming commands.

The library is designed for zero dynamic memory allocation — all buffers are sized at compile time. It runs on a bare-metal main loop with no OS requirement.

1.1 What the Library Does (and Does Not Do)

The library handles: edge timing classification (one vs. zero bits), packet assembly with XOR validation, address matching against CV1/CV17–18/CV29, command dispatch to your callbacks, CV read/write with decoder lock support, service mode ACK pulse generation, failsafe timeout detection, and optional RailCom response encoding.

It does NOT include: motor control, LED/servo/solenoid drivers, sound generation, or any hardware-specific code. You supply driver functions via function pointers.

1.2 Platform Support

Processor / Board	Example IDE / Toolchain
TI MSPM0G3507	Code Composer Studio / Theia
ESP32	Arduino IDE, PlatformIO
STM32 (F4 and others)	STM32 Cube IDE
ATmega328P (Arduino Uno/Nano)	Arduino IDE
Any ARM Cortex-M	GCC + Makefile

The only hardware requirement for a decoder is a GPIO input-capture (edge-detect) interrupt and a microsecond timestamp source.

2. DCC Decoder Concepts

2.1 Signal Reception

The DCC signal on the track rails is a bipolar square wave. The decoder picks up the signal through a bridge rectifier (for power) and an optocoupler or voltage divider (for the logic signal). The logic signal is a series of edges — each transition represents a half-bit boundary.

2.2 Bit Classification

The bit decoder measures the time between consecutive edges. If the interval is less than `DCC_DECODER_HALF_BIT_THRESHOLD_US` (80µs), it is a one-bit half-period. If equal to or greater than 80µs, it is a zero-bit half-period. Two matching half-periods make one complete bit.

Measurement	Classification
Interval < 80 µs	One-bit half-period (58µs nominal)
Interval ≥ 80 µs	Zero-bit half-period (100µs+ nominal)

2.3 Packet Structure

The decoder requires at least 10 consecutive one-bits for a valid preamble (command stations send 14+). After the preamble, a zero start-bit signals the beginning of data. Data bytes are separated by zero start-bits. The packet ends with a one end-bit. The last byte is XOR of all preceding data bytes — if it doesn't match, the packet is discarded.

2.4 Address Matching

The library reads the decoder's address configuration from CVs during `DccConfig_initialize()`:

CV	Purpose
CV1	Primary (short) address, 1–127
CV17–18	Extended (long) address, 128–10239
CV29 bit 5	0 = use short address (CV1), 1 = use long address (CV17–18)
CV29 bit 2	RailCom enable
CV29 bit 1	0 = 14/28-step speed, 1 = 128-step speed
CV29 bit 0	0 = normal direction, 1 = reversed

2.5 Configuration Variables (CVs)

CVs are numbered settings stored in the decoder. The NMRA defines standard meanings for many CV numbers. CVs can be read and written in service mode (programming track) or operations mode (POM, on the main track).

CV	Name	Default
1	Primary address	3
2	Start voltage (Vstart)	0
3	Acceleration rate	0
4	Deceleration rate	0
5	Maximum speed (Vhigh)	0
7	Version number	Manufacturer defined
8	Manufacturer ID (write 8 = factory reset)	Manufacturer defined
17–18	Extended address	0
28	RailCom configuration	0
29	Configuration register	0x06

3. Project File Structure

decoder/	<- Your project folder
decoder.c	<- Main entry (setup + loop + edge ISR)
dcc_user_config.h	<- REQUIRED: feature flags & limits
decoder_command_parser.c/h	<- Demo UART CLI (ADDR, HELP, etc.)
application_callbacks/	
callbacks_dcc.c/h	<- Your motor/LED/servo logic
application_drivers/	
ti_driverlib_dcc_driver.c/h	<- Hardware interface (timestamp, GPIO)
ti_driverlib_uart_driver.c/h	<- UART I/O
ack_pulse_driver.c/h	<- ACK current load control
dcc_lib/	<- Library core (do not edit)
dcc_config.h/c	- Stack init API
dcc_types.h	- All data types
dcc_defines.h	- NMRA constants
dcc_bit_decoder.h/c	- Edge timing classification
dcc_packet_decoder.h/c	- Packet assembly & dispatch
dcc_cv_storage.h/c	- CV read/write with lock
dcc_railcom_encoder.h/c	- RailCom 4/8 response encoding
dcc_application_*.h/c	- Internal application logic

4. dcc_user_config.h in Depth

The decoder configuration is simpler than the command station — only the role flag and function count are needed.

```
// --- Role Selection ---
#define DCC_COMPILE_DECODER           // Bit decoder, packet parser, CV storage
// #define DCC_COMPILE_ACCESSORY_DECODER // Accessory-specific features

// --- Decoder Configuration ---
#define USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS 29 // F0-F28
```

Define	Value	Description
DCC_COMPILE_DECODER	defined	Enables bit decoder, packet decoder, CV storage
DCC_COMPILE_ACCESSORY_DECODER	optional	Adds accessory-specific address handling
USER_DEFINED_DCC_DECODER_MAX_FUNCTIONS	29	F0–F28 (29 functions). Increase for F29–F68.

5. Initialization — decoder.c

5.1 The dcc_config_t Struct (Decoder Fields)

```
const dcc_config_t dcc_config = {
    // REQUIRED: common platform drivers
    .lock_shared_resources    = &TI_DccDriver_lock_shared_resources,
    .unlock_shared_resources = &TI_DccDriver_unlock_shared_resources,
    .get_timestamp_usec       = &TI_DccDriver_get_timestamp_usec,

    // REQUIRED: CV storage
    .cv_read                  = &CallbacksDcc_cv_read,
    .cv_write                 = &CallbacksDcc_cv_write,

    // Service mode ACK pulse
    .start_ack_pulse          = &AckPulseDriver_start,
    .stop_ack_pulse           = &AckPulseDriver_stop,

    // Application callbacks (NULL = no notification)
    .on_speed_command         = &CallbacksDcc_on_speed_command,
    .on_function_command      = &CallbacksDcc_on_function_command,
    .on_failsafe_entered     = &CallbacksDcc_on_failsafe_entered,
    // ... more callbacks ...
};
```

5.2 Edge Timestamp Ring Buffer

The GPIO ISR captures only a timestamp and stores it in a power-of-2 ring buffer. The main loop drains the buffer and feeds timestamps to `DccConfig_decoder_edge_isr()`. This keeps the ISR under $\sim 1\mu\text{s}$ so it never overruns the $58\mu\text{s}$ half-bit period.

```
#define EDGE_BUF_SIZE 256          /* must be power of 2 */
#define EDGE_BUF_MASK (EDGE_BUF_SIZE - 1)

static volatile uint32_t _edge_buf[EDGE_BUF_SIZE];
static volatile uint16_t _edge_head; /* written by ISR only */
static volatile uint16_t _edge_tail; /* read by main loop only */

void GROUP1_IRQHandler(void) {
    uint32_t ts = TI_DccDriver_get_timestamp_usec();
    uint16_t next = (_edge_head + 1) & EDGE_BUF_MASK;
    if (next != _edge_tail) {
        _edge_buf[_edge_head] = ts;
        _edge_head = next;
    }
    // clear interrupt flags...
}
```

5.3 Setup and Main Loop

```
int main(void) {
    SYSCFG_DL_init();           // MCU init
    TI_DccDriver_initialize();   // Timestamp timer
    TI_UartDriver_initialize();  // UART
    AckPulseDriver_initialize(); // ACK current load
    CallbacksDcc_initialize();   // CV defaults
    DccConfig_initialize(&dcc_config); // Library init (reads CVs)
```

```

while (1) {
    _drain_edge_buffer();           // Feed edges to bit decoder
    DccConfig_run();               // Library housekeeping
    CallbacksDcc_drain();          // Push RECV lines to UART
    TI_UartDriver_echo_process();
    DecoderCommandParser_process();
}
}

```

_drain_edge_buffer() must be called frequently. If the ring buffer overflows, edges are silently dropped and packets may be missed.

6. ISR Architecture

6.1 GPIO Edge Interrupt

The primary ISR fires on every rising and falling edge of the DCC input signal. Its only job is to capture a microsecond timestamp into the ring buffer. No decoding, no packet processing — just timestamp and return. This keeps ISR execution under 1 μ s.

The demo supports two physical inputs (PB1 for main track, PB4 for service track) with a track-select mux pin (PB17). Both are on the same GPIO port so they share one ISR.

6.2 100ms Periodic Tick

A 100ms timer (SysTick in the demo) provides a heartbeat. The decoder uses this for LED toggling. The library does not currently require a periodic tick for the decoder role, but it is good practice to have one for watchdog and failsafe timing.

7. Implementing the Drivers

7.1 Platform Drivers

Function	Context	Contract
lock_shared_resources	Any	Disable interrupts or acquire mutex
unlock_shared_resources	Any	Re-enable interrupts or release mutex
get_timestamp_usec	Any	Return monotonic microsecond counter

7.2 CV Storage Drivers

The library reads and writes CVs through `cv_read()` and `cv_write()` function pointers. These are **REQUIRED** — the library reads CV1/CV17–18/CV29 during initialization to determine the decoder's address.

```
bool CallbacksDcc_cv_read(uint16_t cv_number, uint8_t *value) {
    if (cv_number >= CV_ARRAY_SIZE) return false;
    *value = cv_storage[cv_number];
    return true;
}

bool CallbacksDcc_cv_write(uint16_t cv_number, uint8_t value) {
    if (cv_number >= CV_ARRAY_SIZE) return false;
    cv_storage[cv_number] = value;
    return true;
}
```

The demo stores CVs in a RAM array that resets on power cycle. For a real product, replace with Flash or EEPROM read/write. The `cv_write` function should write to non-volatile storage.

7.3 ACK Pulse Driver

During service mode programming, the decoder must respond with a 6ms current pulse (ACK) when a command succeeds. The library calls `start_ack_pulse()` to turn on the current load and `stop_ack_pulse()` to turn it off (after 6ms, timed by the library). Set both to NULL if your decoder does not support service mode.

8. The Bit Decoder

The bit decoder receives raw edge timestamps from `DccConfig_decoder_edge_isr()` and classifies each pair of half-periods into one-bits or zero-bits.

For each edge, it calculates the interval since the previous edge. If the interval is less than `DCC_DECODER_HALF_BIT_THRESHOLD_US` ($80\mu\text{s}$), it is a short half-period (one-bit). If $\geq 80\mu\text{s}$, it is a long half-period (zero-bit). Two consecutive matching half-periods form one complete bit. If the two halves disagree (one short + one long), the bit is invalid and the decoder resets to preamble search.

The NMRA allows stretching the first half of a zero-bit up to $12000\mu\text{s}$ for decoder processing time. The bit decoder accommodates this by accepting any long half-period as a potential zero-bit first half.

9. The Packet Decoder

The packet decoder operates as a state machine with these states:

State	Description
PREAMBLE	Counting consecutive one-bits (need ≥ 10)
START_BIT	Waiting for the first zero-bit after preamble
DATA	Collecting 8 data bits into a byte
END_OR_START	After each byte: one-bit = end of packet, zero-bit = start of next byte

When a complete packet is received, the decoder validates the XOR error check byte. If valid, it checks the address against the decoder's configured address (from CVs). Broadcast packets (address 0) are always accepted. If the address matches, the decoder parses the instruction byte(s) and dispatches to the appropriate callback.

10. CV Storage and Address Matching

The library maintains an internal shadow copy of the critical CVs (CV1, CV17–18, CV29) for fast address matching. When a CV write command changes one of these CVs, the library updates its shadow copy automatically.

CV8 has special behavior: writing the value 8 to CV8 triggers a factory reset. The library calls your `cv_write` function to reset all CVs to their default values.

Decoder Lock

The library supports NMRA decoder lock (CV15/CV16). When locked, CV write commands are ignored unless the lock value matches. This prevents accidental programming of the wrong decoder when multiple decoders share the same address on the programming track.

11. Service Mode (Decoder Side)

When the decoder is on the programming track and receives service mode packets, the library handles the protocol automatically:

1. The library recognizes the service mode packet sequence (reset + command packets).
2. For write commands: the library calls `cv_write()` to store the value, then calls `start_ack_pulse()` to trigger the 6ms ACK current pulse.
3. For verify commands: the library calls `cv_read()` and compares the value. If it matches, it triggers the ACK pulse.
4. For bit operations: same as verify/write but at the individual bit level.
5. The library calls `stop_ack_pulse()` after 6ms (`DCC_ACK_PULSE_DURATION_US`).

Your `on_cv_write_command` and `on_cv_verify_command` callbacks fire after the operation completes, giving you a chance to log or react to CV changes.

12. RailCom Responses (Brief Overview)

If `railcom_tx_pin_set` is non-NULL, the library can send RailCom responses back to the command station during the cutout window. The library bit-bangs 4/8-encoded bytes at 250 kbaud (4µs per bit) through a GPIO connected to a current-source circuit.

The decoder must also blank the edge-detect interrupt during the cutout to avoid false edges. Set `decoder_edge_irq_enable` to a function that enables/disables the GPIO interrupt.

Config Field	Purpose
<code>railcom_tx_pin_set(high)</code>	Drive GPIO high/low for RailCom current-source
<code>decoder_edge_irq_enable(enabled)</code>	Blank edge-detect during cutout window
<code>railcom_uart_write</code>	Alternative: send via hardware UART (NULL in most cases)

13. Failsafe Mode

The NMRA requires decoders to enter a safe state if no valid DCC packet is received for a defined period. The library monitors the time since the last valid packet. If it exceeds the failsafe timeout, it calls `on_failsafe_entered()`. When valid packets resume, it calls `on_failsafe_exited()`.

In your `on_failsafe_entered` callback, you should stop the motor and set all outputs to a safe state. Possible causes: track power turned off, wiring fault, decoder fell off the track, or command station crashed.

```
void CallbacksDcc_on_failsafe_entered(void) {
    set_motor_speed(0, true); // Stop motor
    set_all_lights(false);    // Lights off
}

void CallbacksDcc_on_failsafe_exited(void) {
    // Normal operation resumes.
    // Speed/function commands will arrive shortly.
}
```

14. Callbacks — Where Your Application Lives

All your application logic lives in `callbacks_dcc.c`. Each callback fires when the library decodes the corresponding DCC command. The demo logs to UART; replace with your motor/LED/servo control.

Callback	Signature	When It Fires
<code>on_speed_command</code>	<code>(addr, speed, dir, mode)</code>	Speed + direction
<code>on_emergency_stop_command</code>	<code>(addr)</code>	E-stop for this address
<code>on_function_command</code>	<code>(addr, func_num, state)</code>	Function on/off (F0–F68)
<code>on_accessory_basic_command</code>	<code>(board, pair, activate)</code>	Turnout throw/close
<code>on_accessory_extended_command</code>	<code>(addr, aspect)</code>	Signal aspect
<code>on_cv_write_command</code>	<code>(cv, value, svc_mode)</code>	CV was written
<code>on_cv_verify_command</code>	<code>(cv, value, svc_mode)</code>	CV verify received
<code>on_cv_bit_command</code>	<code>(cv, bit, value, svc_mode)</code>	CV bit manipulation
<code>on_consist_command</code>	<code>(addr, ca, dir_normal)</code>	Consist setup/clear
<code>on_binary_state_short</code>	<code>(addr, state, active)</code>	Binary state (1–127)
<code>on_binary_state_long</code>	<code>(addr, state, active)</code>	Binary state (1–32767)
<code>on_analog_function</code>	<code>(addr, output, value)</code>	Analog output (0–255)
<code>on_failsafe_entered</code>	<code>()</code>	No valid DCC packets
<code>on_failsafe_exited</code>	<code>()</code>	Valid DCC resumed

15. Porting to a New MCU

Porting the decoder to a new MCU requires these steps:

1. Copy the decoder example project.
2. Replace `ti_driverlib_dcc_driver.c/h` with your MCU's GPIO and timer API.
3. Replace `ack_pulse_driver.c/h` with your MCU's current load control.
4. Replace `ti_driverlib_uart_driver.c/h` for debug output (optional).
5. Wire the GPIO edge-detect ISR to capture timestamps into the ring buffer.
6. Provide a microsecond timestamp source (hardware timer or DWT cycle counter).
7. Build and test with a known-good command station.

Minimum hardware: one GPIO with edge-detect interrupt, one microsecond timer or counter. Optional: GPIO for ACK current load, GPIO for RailCom current-source.

16. Unit Testing

The decoder modules have comprehensive unit tests:

Test File	What It Tests
dcc_bit_decoder_Test.cxx	Edge timing classification, threshold handling, noise rejection
dcc_packet_decoder_Test.cxx	Preamble detection, byte assembly, XOR validation, address matching
dcc_cv_storage_Test.cxx	CV read/write, decoder lock, factory reset
dcc_railcom_encoder_Test.cxx	4/8 encoding for RailCom responses

```
cd test
mkdir build && cd build
cmake ..
make
ctest --output-on-failure
```

17. Troubleshooting

Symptom	Likely Cause and Fix
No packets decoded	Check DCC input wiring. Verify GPIO edge-detect interrupt is enabled. Check ring buffer is being drained.
Packets decoded but callbacks not firing	Address mismatch. Check CV1/CV29 defaults in <code>CallbacksDcc_initialize()</code> . Try broadcast address 0.
Corrupted packets (XOR errors)	Noise on DCC input. Add filtering/debounce. Check signal levels (must be clean 3.3V logic).
ACK not detected by command station	Check ACK current load circuit. Verify 6ms pulse is being generated. Check current threshold on command station side.
Failsafe fires immediately	No DCC signal reaching the decoder. Check wiring and track power.
Functions work but speed doesn't	CV29 speed mode mismatch. Check CV29 bit 1 (0 = 14/28 step, 1 = 128 step) matches command station.
Undefined reference to <code>USER_DEFINED_*</code>	<code>dcc_user_config.h</code> not on include path. Add <code>-I</code> flag.
Wrong address after power cycle	CV storage is RAM-only (demo default). Replace with Flash/EEPROM for persistence.

Online Resources

Library source: github.com/jimkueneman/OpenDccCLib

NMRA DCC standards: www.nmra.org/dcc